

Question Bank — Week 8

Searching (Lecture 08) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Answers are in a separate file: [week08-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. The functions are those of `dsa/searching.py`. `binary_search` uses an inclusive range `values[Lo..hi]` and `mid = (Lo + hi) // 2`; `Lower_bound` and `upper_bound` use a half-open range `[Lo, hi)` starting at `Lo = 0`, `hi = Len(values)`. Unless a question says otherwise, **V** is the sorted list

`V = [3, 6, 8, 12, 15, 19, 21, 24, 27, 31, 34, 38, 41, 45, 48, 52]`
of 16 values, at indices 0 to 15.

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W8-M01 – W8-M22
B	Short answer and essay	W8-E1 – W8-E5
C	Trace the algorithms	W8-T1 – W8-T5
D	Complexity analysis	W8-K1 – W8-K3
E	Find and fix the bug	W8-B1 – W8-B4
F	Write the code — checked by pytest	W8-C1 – W8-C5

Part A — Multiple choice

W8-M01 [what] Which search works on an **unsorted** list?

- **a)** linear search
- **b)** binary search
- **c)** jump search
- **d)** interpolation search

W8-M02 [how] The largest number of comparisons binary search makes on a sorted list of 1,000 elements is:

- **a)** 1,000
- **b)** 500
- **c)** 10
- **d)** 32

W8-M03 [how] ...and on 1,000,000 elements:

- **a)** 1,000
- **b)** 20
- **c)** 1,000,000
- **d)** 6

W8-M04 [why] Binary search is called on a list that is **not** sorted. It:

- **a)** raises an error
- **b)** sorts the list first
- **c)** always returns -1
- **d)** may return a wrong answer, with no error

W8-M05 [how] `binary_search(V, 20)`: the index of the **first** element compared is:

- **a)** 7
- **b)** 8
- **c)** 3
- **d)** 15

W8-M06 [how] ...and the index of the **second**:

- **a)** 11
- **b)** 5
- **c)** 3
- **d)** 6

W8-M07 [what] The loop invariant of binary search is:

- **a)** `lo` is always smaller than `mid`
- **b)** if the target is anywhere in the list, it is in `values[lo..hi]`
- **c)** `values[mid]` is always the target
- **d)** the range halves exactly at every step

W8-M08 [why] A binary search with an inclusive `hi` loops while `lo < hi`. Searching `[42]` for 42, it returns:

- **a)** 0
- **b)** an `IndexError`
- **c)** nothing — it loops for ever
- **d)** -1

W8-M09 [why] In Java, `mid = (lo + hi) / 2` is a bug because:

- **a)** `lo + hi` can overflow a 32-bit `int` on very large arrays
- **b)** Java has no integer division
- **c)** it rounds the wrong way
- **d)** it is slower than `lo + (hi - lo) / 2`

W8-M10 [why] A recursive binary search calls itself on `values[:mid]` or `values[mid + 1:]`. Its total time on n elements is:

- **a)** $O(\log n)$

- **b)** $O(1)$
- **c)** $O(n)$ — the slices copy
- **d)** $O(n \log n)$

W8-M11 [what] The extra space of `binary_search_recursive` (passing `lo` and `hi`, no slices) is:

- **a)** $O(1)$
- **b)** $O(n)$
- **c)** $O(n \log n)$
- **d)** $O(\log n)$ — the call stack

W8-M12 [how] `lower_bound([1, 2, 2, 2, 5, 7], 2)` is:

- **a)** 0
- **b)** 1
- **c)** 3
- **d)** 4

W8-M13 [how] `upper_bound([1, 2, 2, 2, 5, 7], 2)` is:

- **a)** 4
- **b)** 3
- **c)** 1
- **d)** 5

W8-M14 [how] `lower_bound([1, 2, 2, 2, 5, 7], 8)` is:

- **a)** -1
- **b)** 5
- **c)** an error
- **d)** 6

W8-M15 [how] How many times does 3 occur in `[1, 3, 3, 3, 3, 9]`, computed as `upper_bound(v, 3) - lower_bound(v, 3)`?

- **a)** 3
- **b)** 5
- **c)** 4
- **d)** 1

W8-M16 [what] Python's `bisect.bisect_left(values, x)` is the same as:

- **a)** `binary_search`
- **b)** `lower_bound`
- **c)** `upper_bound`
- **d)** `linear_search`

W8-M17 [how] Jump search on 100 sorted elements jumps by:

- **a)** 50
- **b)** 2
- **c)** 7

- **d)** 10

W8-M18 [why] Jump search uses blocks of size $\sqrt{n}$ because:

- **a)** it minimises the total $n/m + m$ of jumps plus steps inside a block
- **b)** $\sqrt{n}$ is always an integer
- **c)** smaller blocks would be incorrect
- **d)** it makes jump search $O(\log n)$

W8-M19 [how] Exponential search for a target at index 11 of a 32-element list checks the bounds:

- **a)** 1, 3, 7, 15
- **b)** 11 only
- **c)** 1, 2, 4, 8, 16
- **d)** 16, 8, 4, 2, 1

W8-M20 [why] Exponential search is most useful when:

- **a)** the list is unsorted
- **b)** the length is unknown, or the target is near the front
- **c)** the values are evenly spread
- **d)** the list has duplicates

W8-M21 [why] The worst case of interpolation search is:

- **a)** $O(\log \log n)$
- **b)** $O(\log n)$
- **c)** $O(\sqrt{n})$
- **d)** $O(n)$ — on skewed values

W8-M22 [how] Interpolation search on [10, 20, 30, ..., 100] (10 values) for 70: the first index it probes is:

- **a)** 6
 - **b)** 4
 - **c)** 7
 - **d)** 5
-

Part B — Short answer and essay

W8-E1 [why] (4 marks) Describe binary search. State its loop invariant and use it to explain why the algorithm is correct, and show that it makes at most $\lfloor \log_2 n \rfloor + 1$ comparisons. What must the caller guarantee?

W8-E2 [how] (4 marks) Give four classic bugs in binary search. For each, give an input on which it fails, what happens, and the fix.

W8-E3 [why] (4 marks) Define `lower_bound` and `upper_bound`. Give three uses of them, and explain why they never return `-1`.

W8-E4 [why] (3 marks) Compare jump search, exponential search and interpolation search: cost, and a situation in which each is the right choice.

W8-E5 [why] (3 marks) A program searches an unsorted list of n values k times. When is it worth sorting the list first? Compare the costs, and give a concrete example with $n = 1,000,000$.

Part C — Trace the algorithms

W8-T1 [how] Trace `binary_search` on `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]` for 23, and then for 60, as tables of `lo`, `hi`, `mid` and `values[mid]`. Where does `lo` end up when 60 is not found, and what does that position mean?

W8-T2 [how] Trace `lower_bound` and `upper_bound` on `[1, 2, 2, 2, 5, 7]` for 2, as tables of `lo`, `hi`, `mid` and the direction taken.

W8-T3 [how] Draw the calls of `binary_search_recursive(V, 45)` and `binary_search_recursive(V, 7)`: the `lo` and `hi` of each call, and what it returns. What is the depth of the deeper one?

W8-T4 [how] Trace `jump_search` on the 25 odd numbers `[1, 3, 5, ..., 49]` for 30. Which elements are read, and what is returned?

W8-T5 [why] Trace `interpolation_search` for 70 and for 75 on `[10, 20, ..., 100]`, and for 9 on `[1, 2, 3, 4, 5, 6, 7, 8, 9, 1000]`. Count the probes. What does the last case show?

Part D — Complexity analysis

W8-K1 [how] Give Θ and justify, for a sorted list a of n values and a list b of m values:

```
found = 0
for x in b:
    if binary_search(a, x) != -1:
        found += 1
```

What is it with `linear_search`? And if a were not sorted?

W8-K2 [how] Write the recurrence and give Θ for a recursive binary search that passes **slices** (`values[mid + 1:]`) instead of `lo` and `hi`.

W8-K3 [why] A student counts the copies of x in a sorted list: find **any** copy with `binary_search`, then walk left and right while the neighbours equal x . Give its cost in terms of n and the number of copies c , and its worst case. What is the $O(\log n)$ way?

Part E — Find and fix the bug

W8-B1 [how]

```
def binary_search(values, target):
    lo, hi = 0, len(values) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if values[mid] == target:
            return mid
        if values[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

W8-B2 [why]

```
def binary_search(values, target):
    lo, hi = 0, len(values) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if values[mid] == target:
            return mid
        if values[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return -1
```

W8-B3 [why]

```
def binary_search_recursive(values, target):
    if not values:
        return -1
    mid = len(values) // 2
    if values[mid] == target:
        return mid
    if values[mid] < target:
        return binary_search_recursive(values[mid + 1:], target)
    return binary_search_recursive(values[:mid], target)
```

W8-B4 [how] Part of interpolation_search:

```
lo, hi = 0, len(values) - 1
while lo <= hi and values[lo] <= target <= values[hi]:
    pos = lo + (target - values[lo]) * (hi - lo) // (values[hi] - values[lo])
    ...
```

Part F — Write the code

In `practice/week08.py`; check with `pytest tests/test_practice_week08.py -v`. Each problem must be $O(\log n)$ (C5: $O(n \log S)$): the tests count your reads. Do not use `bisect`, `in`, `index` or `sorted`.

W8-C1 [why] `search_rotated(values, target)` — binary search on a sorted list that has been rotated. Why is one half always sorted?

W8-C2 [how] `integer_sqrt(n)` — the largest r with $r^2 \leq n$, by binary search on the answer.

W8-C3 [why] `find_peak(values)` — an element not smaller than its neighbours, in $O(\log n)$, on an **unsorted** list. Why is it safe to discard half?

W8-C4 [how] `closest_value(values, target)` — the nearest value, ties to the smaller.

W8-C5 [why] `min_capacity(weights, days)` — the smallest ship capacity that delivers the packages in time. Why may you binary search on the capacity?